

Grupo de trabajo: Sabrina Gimenez, Ismael Saleme, Jeronimo Coronel y Joaquin godoy

Base de Datos II — Unidad 4: Forma Normal de Boyce-Codd y Desnormalización Controlada en Food Store

Base de trabajo: Food_Store_Copia (PostgreSQL 17) — la base original Food_Store no fue modificada.

Fecha: octubre de 2026

Repositorio: scripts en db/tp_fnbc_control_lote.sql y db/tp_desnormalizacion_top_categorias.sql

1. Parte 1 — Análisis y aplicación de la FNBC: ControlLoteAlmacen

Food Store extiende su operación a la distribución mayorista: recibe lotes de mercadería que se controlan (calidad) desde uno o más depósitos. Por cada par (lote, depósito) se designa un responsable de control, y cada responsable pertenece a un único depósito.

Esquema inicial e instancia de ejemplo (enunciado 4.1)

```
CREATE TABLE control_lote_almacen (
    lote_id          BIGINT NOT NULL REFERENCES lote(id),
    deposito_id      BIGINT NOT NULL REFERENCES deposito(id),
    responsable_control_id BIGINT NOT NULL REFERENCES usuario(id),
    PRIMARY KEY (lote_id, deposito_id)
);
INSERT INTO control_lote_almacen VALUES (501, 30, 801), (502, 30, 801), (503, 31, 802);
```

a) Dependencias funcionales (notación formal)

Sea el esquema $R(\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID})$. De la regla de negocio se desprenden:

- $F1: \{\text{LoteID}, \text{DepositoID}\} \rightarrow \text{ResponsableControlID}$ — para un lote y un depósito dados, el responsable queda unívocamente determinado.
- $F2: \text{ResponsableControlID} \rightarrow \text{DepositoID}$ — cada responsable pertenece a un único depósito (dato maestro de personal).

b) Clausura de atributos y claves candidatas

Clausuras calculadas con $F = \{F1, F2\}$:

Conjunto X	Clausura X^+	¿Superclave?
$\{\text{LoteID}\}^+$	$\{\text{LoteID}\}$	No
$\{\text{DepositoID}\}^+$	$\{\text{DepositoID}\}$	No
$\{\text{ResponsableControlID}\}^+$	$\{\text{ResponsableControlID}, \text{DepositoID}\}$ (por F2)	No (falta LoteID)
$\{\text{LoteID}, \text{DepositoID}\}^+$	$\{\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID}\}$ (por F1)	Sí — mínima: candidata
$\{\text{LoteID}, \text{ResponsableControlID}\}^+$	$\{\text{LoteID}, \text{ResponsableControlID}, \text{DepositoID}\}$ ($R \rightarrow D$ agrega	Sí — mínima: candidata

	DepositoID)	
{DepositoID, ResponsableControlID}*	{DepositoID, ResponsableControlID}	No (falta LoteID)

Claves candidatas: {LoteID, DepositoID} y {LoteID, ResponsableControlID}. Atributos primos: LoteID, DepositoID y ResponsableControlID (los tres pertenecen a alguna clave candidata). No hay atributos no primos.

c) Demostración de violación de la FNBC (definición formal)

Definición formal de FNBC: un esquema R está en FNBC si y solo si para toda dependencia funcional no trivial $X \rightarrow A$ que se cumple en R, el determinante X es superclave de R.

La dependencia F2: ResponsableControlID $\rightarrow$ DepositoID es no trivial ($\text{DepositoID} \notin \{\text{ResponsableControlID}\}$) y se cumple en R, pero su determinante NO es superclave: $\{\text{ResponsableControlID}\}^+ = \{\text{ResponsableControlID}, \text{DepositoID}\} \neq \{\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID}\}$. Por lo tanto control_lote_almacen NO está en FNBC; la dependencia violatoria es F2.

Observación: como los tres atributos son primos, toda DF tiene consecuente primo y el esquema sí cumple 3FN. Es el caso clásico donde 3FN no alcanza y FNBC exige descomponer.

d) Anomalías clásicas sobre la instancia de ejemplo

Anomalía de inserción: si ingresa un nuevo responsable (p. ej. 803) asignado al depósito 30 pero aún sin ningún lote que controlar, es imposible registrar el hecho “803 pertenece al depósito 30”: la PK (lote_id, deposito_id) y los NOT NULL obligarían a inventar un lote ficticio.

Anomalía de borrado: la fila (503, 31, 802) es la única que vincula al responsable 802 con el depósito 31; si ese control se elimina (p. ej. se anula el lote 503), se pierde para siempre el dato maestro “802 pertenece al depósito 31”.

Anomalía de actualización: el hecho “801 pertenece al depósito 30” está redundante en dos filas (501 y 502); si 801 se traslada al depósito 32 hay que actualizar N filas y, ante una actualización parcial, la base queda inconsistente (un lote afirma 30 y otro 32 para el mismo responsable).

e) Descomposición sin pérdida (script SQL)

Aplicado el algoritmo sobre la violadora F2 (X = responsable_control_id, Y = deposito_id): $R1 = X \cup Y$ y $R2 = R - Y$. Tablas resultantes (ver db/tp_fnbc_control_lote.sql):

```
CREATE TABLE responsable_deposito (
    responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),
    deposito_id            BIGINT NOT NULL REFERENCES deposito(id)
);
CREATE TABLE control_lote (
    lote_id                BIGINT NOT NULL REFERENCES lote(id),
    responsable_control_id BIGINT NOT NULL REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id)
);
CREATE OR REPLACE VIEW v_control_lote_almacen AS
```

```
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
FROM control_lote cl
JOIN responsable_deposito rd ON rd.responsable_control_id = cl.responsable_control_id;
```

f) Justificación de la unión sin pérdida y migración verificada

Criterio de superclave sobre el atributo común: el atributo común de la descomposición es responsable_control_id, que es clave primaria —y por ende superclave— de responsable_deposito (R1). En consecuencia, cada fila de control_lote (R2) se combina con exactamente una fila de R1: la reunión no puede generar tuplas espurias ni perder tuplas. Es sin pérdida.

Migración: INSERT INTO responsable_deposito SELECT DISTINCT responsable_control_id, deposito_id FROM control_lote_almacen; INSERT INTO control_lote SELECT lote_id, responsable_control_id FROM control_lote_almacen. Verificación ejecutada sobre Food_Store_Copia:

Control	Resultado
Filas originales sin correlato en la vista (EXCEPT)	0 filas
Filas de la vista no presentes en el original (EXCEPT)	0 filas
Conteos original / vista / responsable_deposito / control_lote	3 / 3 / 2 / 3

2. Parte 2 — Consulta con desnormalización controlada

Adaptaciones documentadas respecto del enunciado 5.1

- detalle_pedido no posee columna subtotal: se utiliza cantidad × precio_unitario, su definición de negocio.
- pedido y detalle_pedido no poseen columna eliminado: no existe borrado lógico en este esquema, por lo que dichos filtros se omiten.
- No existen pedidos con fecha de hoy (rango 2025-09-04 a 2026-09-04, ~500 pedidos/día, 200.000 pedidos totales): el “día” del reporte es el día con datos más reciente (2026-09-04, 404 pedidos), equivalente funcional de CURRENT_DATE sobre esta instancia.

a) Consulta base con EXPLAIN ANALYZE (antes)

```
SELECT c.nombre AS categoria, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido ped ON ped.id = dp.pedido_id
WHERE ped.fecha::date = (SELECT MAX(fecha::date) FROM pedido)
GROUP BY c.nombre ORDER BY total_vendido DESC LIMIT 5;
-- Execution Time: 73.935 ms
```

Salida completa de EXPLAIN (ANALYZE, TIMING ON):

```
Limit (cost=11733.83..11733.84 rows=5 width=210) (actual time=69.729..73.728 rows=5 loops=1)
  InitPlan 1
    -> Finalize Aggregate (cost=4235.82..4235.83 rows=1 width=4) (actual time=28.486..28.511 rows=1
loops=1)
```

```

-> Gather (cost=4235.71..4235.82 rows=1 width=4) (actual time=27.802..28.506 rows=2 loops=1)
    -> Partial Aggregate (cost=3235.71..3235.72 rows=1 width=4) (actual
time=13.751..13.752 rows=1 loops=2)
        -> Parallel Seq Scan on pedido (cost=0.00..2647.47 rows=117647 width=8) (actual
time=0.011..4.251 rows=100000 loops=2)
            -> Sort (cost=7498.00..7498.92 rows=370 width=210) (actual time=69.728..73.701 rows=5 loops=1)
                -> Finalize GroupAggregate (cost=7430.85..7491.85 rows=370 width=210) (actual
time=69.391..73.424 rows=10 loops=1)
                    -> Gather Merge (cost=7430.85..7484.45 rows=370 width=210) (actual time=69.377..73.412
rows=20 loops=1)
                        -> Partial GroupAggregate (cost=6430.84..6442.82 rows=370 width=210) (actual
time=26.212..26.262 rows=10 loops=2)
                            -> Sort (cost=6430.84..6432.31 rows=588 width=188) (actual
time=25.697..25.705 rows=202 loops=2)
                                -> Nested Loop (cost=3243.50..6403.80 rows=588 width=188) (actual
time=10.548..25.562 rows=202 loops=2)
                                    -> Nested Loop (cost=3243.35..6387.31 rows=588 width=18) (actual
time=10.410..25.313 rows=202 loops=2)
                                        -> Parallel Hash Join (cost=3243.06..6199.36 rows=588
width=18) (actual time=10.066..21.933 rows=202 loops=2)
                                            Hash Cond: (dp.pedido_id = ped.id)
                                                -> Parallel Seq Scan on detalle_pedido dp
(cost=0.00..2647.47 rows=117647 width=26) (actual time=0.007..4.708 rows=100000 loops=2)
                                                    -> Parallel Hash (cost=3235.71..3235.71 rows=588
width=8) (actual time=9.975..9.976 rows=202 loops=2)
                                                        -> Parallel Seq Scan on pedido ped
(cost=0.00..3235.71 rows=588 width=8) (actual time=0.028..19.866 rows=404 loops=1)
                                                            Filter: ((fecha)::date = (InitPlan
1).col1)
                                                                Rows Removed by Filter: 199596
                                                                    -> Index Scan using producto_pkey on producto pr (actual
time=0.016..0.016 rows=1 loops=404)
                                                                        -> Memoize (actual time=0.001..0.001 rows=1 loops=404)
                                                                            -> Index Scan using categoria_pkey on categoria c (actual
time=0.014..0.014 rows=1 loops=20)
Planning Time: 19.645 ms
Execution Time: 73.935 ms

```

Nodo dominante del costo: los Parallel Seq Scan sobre pedido (el filtro fecha::date no es indexable: lee 200.000 filas y descarta 199.596) y sobre detalle_pedido, más el Parallel Hash Join entre ambas. Mucho trabajo para un resultado final de solo 10 grupos.

b) Patrón elegido y justificación

Patrón elegido: tabla de agregados diarios (resumen_venta_categoria_dia) mantenida por disparador AFTER INSERT/UPDATE/DELETE sobre detalle_pedido. La medición de (a) muestra que el costo lo dominan los recorridos completos de pedido y detalle_pedido por el filtro no indexable fecha::date más los tres JOINS y el GROUP BY, para un resultado de apenas 10 grupos; una vista materializada quedaría obsoleta entre REFRESH y REFRESH justo en un panel requerido “en tiempo real”, mientras que la tabla de agregados se mantiene sincronizada fila a fila por disparadores y es totalmente reversible sin pérdida de información: basta DROP TABLE + DROP TRIGGER porque la fuente de verdad permanece intacta.

c) Implementación (ver db/tp_desnormalizacion_top_categorias.sql)

- resumen_venta_categoria_dia(dia DATE, categoria_id FK, total_vendido NUMERIC, PK(dia, categoria_id)) + backfill histórico: 3.660 filas (366 días × 10 categorías).
- Función fn_sync_resumen_venta_categoria() + trigger trg_sync_resumen_venta por fila: aplica deltas (INSERT suma, DELETE resta, UPDATE reversa el aporte anterior y suma el nuevo).
- Prueba del mecanismo: INSERT de prueba (2 × 1868.53) movió el resumen exactamente +3737.06 y el ROLLBACK posterior dejó la base intacta (200.000 pedidos).

d) Consulta desnormalizada con EXPLAIN ANALYZE (después)

```
SELECT c.nombre AS categoria, r.total_vendido
FROM resumen_venta_categoria_dia r
JOIN categoria c ON c.id = r.categoria_id
WHERE r.dia = (SELECT MAX(dia) FROM resumen_venta_categoria_dia)
ORDER BY r.total_vendido DESC LIMIT 5;
-- Execution Time: 0.232 ms
```

Salida completa de EXPLAIN (ANALYZE, TIMING ON):

```
Limit (cost=46.71..46.73 rows=5 width=196) (actual time=0.181..0.183 rows=5 loops=1)
  InitPlan 2
    -> Result (cost=0.35..0.36 rows=1 width=4) (actual time=0.072..0.073 rows=1 loops=1)
      InitPlan 1
        -> Limit (cost=0.28..0.35 rows=1 width=4) (actual time=0.069..0.070 rows=1 loops=1)
          -> Index Only Scan Backward using resumen_venta_categoria_dia_pkey (actual
time=0.069..0.069 rows=1 loops=1)
        -> Sort (cost=46.36..46.41 rows=19 width=196) (actual time=0.181..0.182 rows=5 loops=1)
          -> Hash Join (cost=31.36..46.04 rows=19 width=196) (actual time=0.147..0.151 rows=10 loops=1)
            -> Seq Scan on categoria c (actual time=0.011..0.012 rows=10 loops=1)
            -> Hash (cost=31.12..31.12 rows=19 width=26) (actual time=0.130..0.130 rows=10 loops=1)
              -> Bitmap Heap Scan on resumen_venta_categoria_dia r (actual time=0.099..0.123
rows=10 loops=1)
                Recheck Cond: (dia = (InitPlan 2).col1)
                -> Bitmap Index Scan on resumen_venta_categoria_dia_pkey (actual
time=0.088..0.088 rows=10 loops=1)
          Planning Time: 2.793 ms
          Execution Time: 0.232 ms
```

Tabla comparativa antes / después:

	Antes (normalizada)	Después (desnormalizada)
Tiempo de ejecución	73.935 ms	0.232 ms (≈319× más rápido)
Nodo dominante	Parallel Seq Scan + Hash Join (200.000 filas)	Bitmap Index Scan sobre PK (10 filas)
Top-5 obtenido	Comidas 409774.42, Postres 393148.97, Limpieza 390863.60, Verdulería 379910.58, Lácteos 358138.75	Idéntico al anterior (mismo reporte)

e) Auditoría de sincronía (resultado vacío)

Script: FULL JOIN entre el resumen y el GROUP BY recalculado desde la fuente de verdad, filtrando diferencias (IS DISTINCT FROM). Ejecutado sobre la base migrada: 0 filas — el dato redundante está sincronizado.

3. Conclusión y entregables

- tp_fnbc_control_lote.sql — esquema original, instancia, tablas descompuestas, vista de compatibilidad y migración verificada.
- tp_desnormalizacion_top_categorias.sql — estructura desnormalizada, sincronización, consulta 5.2(d) y auditoría 5.2(e).
- El presente informe — DFs, claves, violación FNBC, unión sin pérdida, EXPLAIN antes/después y justificación del patrón.